

LAB ASSIGNMENT - 4

System Calls

1. Process Management System Calls

(i) fork() -

- A new process is created by the fork() system call.
- A new process may be created with fork() without a new program being run-the new sub-process simply continues to execute exactly the same program that the first (parent) process was running.
- It is one of the most widely used system calls under process management.
- Creates a child process that runs concurrently with the parent process. It returns:
 - 0 to the child process
 - Child's PID to the parent process
 - -1 if creation fails

Example:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    pid_t pid = fork();
    if (pid == 0)
        printf("Child process\n");
    else
        printf("Parent process\n");
    return 0;
}
```

(ii) exec() -

- A new program will start executing after a call to exec()
- Running a new program does not require that a new process be created first: any process may call exec() at any time. The currently running program is immediately terminated, and the new program starts executing in the context of the existing process.
- Replaces the current process image with a new process image.

Example;

```
#include <unistd.h>

int main() {
    char *args[] = {"ls", "-l", NULL};
    execvp("ls", args);
    return 0;
}
```

(iii) wait() -

- Makes the parent process wait until a child process terminates.

Example:

```
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    if (fork() == 0)
        execlp("ls", "ls", NULL);
    else {
        wait(NULL);
        printf("Child process finished\n");
    }
    return 0;
}
```

(iv) exit -

- The exit() system call is used by a program to terminate its execution.
- The operating system reclaims resources that were used by the process after the exit() system call.

Example:

```
#include <stdlib.h>

int main() {
    exit(0);
}
```

2. File Management System Calls

(i) open() -

- It is the system call to open a file.
- This system call just opens the file, to perform operations such as read and write, we need to execute different system call to perform the operations.

Example:

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);
    close(fd);
    return 0;
}
```

(ii) read() -

- This system call opens the file in reading mode
- We can not edit the files with this system call.
- Multiple processes can execute the read() system call on the same file simultaneously.

Example:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    char buf[100];
    int fd = open("file.txt", O_RDONLY);
    int n = read(fd, buf, 100);
    write(1, buf, n); // 1 = stdout
    close(fd);
    return 0;
}
```

(iii) write() -

- This system call opens the file in writing mode
- We can edit the files with this system call.
- Multiple processes can not execute the write() system call on the same file simultaneously.

Example:

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_WRONLY | O_CREAT, 0644);
    write(fd, "Hello, File!", 12);
    close(fd);
    return 0;
}
```

(iv) close() -

- This system call closes the opened file.

Example:

```
int fd = open("file.txt", O_RDONLY);  
close(fd);
```

3. Device Management System calls

(i) read() -

- This system call opens the file in reading mode
- We can not edit the files with this system call.
- Multiple processes can execute the read() system call on the same file simultaneously.

Example:

```
#include <fcntl.h>  
#include <unistd.h>  
#include <stdio.h>  
  
int main() {  
    char buf[100];  
    int fd = open("file.txt", O_RDONLY);  
    int n = read(fd, buf, 100);  
    write(1, buf, n); // 1 = stdout  
    close(fd);  
    return 0;  
}
```

(ii) write() -

- This system call opens the file in writing mode
- We can edit the files with this system call.
- Multiple processes can not execute the write() system call on the same file simultaneously.

Example:

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("file.txt", O_WRONLY | O_CREAT, 0644);
    write(fd, "Hello, File!", 12);
    close(fd);
    return 0;
}
```

(iii) ioctl() -

- ioctl() is referred to as Input and Output Control.
- ioctl is a system call for device-specific input/output operations and other operations which cannot be expressed by regular system calls.

Example:

```
#include <sys/ioctl.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int result = ioctl(0, TCGETS, NULL); // Get terminal attributes from stdin
    printf("Result: %d\n", result);
    return 0;
}
```

(iv) select() -

- Monitors multiple file descriptors to see if I/O is possible.

Example:

```
#include <sys/select.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    fd_set set;
    FD_ZERO(&set);
    FD_SET(0, &set); // stdin
    select(1, &set, NULL, NULL, NULL);
    printf("Input detected\n");
    return 0;
}
```


4. Network Management System Calls

(i) socket() -

- Creates a new socket.

Example:

```
#include <sys/socket.h>

int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

(ii) connect() -

- Establishes a connection to a server.

Example:

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>

struct sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

(iii) send() -

- Used to send messages on a socket.

Example:

```
send(sockfd, "Hello", 5, 0);
```

(iv) recv() -

- Used to receive messages on a socket.

Example:

```
recv(sockfd, buffer, sizeof(buffer), 0);
```

5. System Information Management System Calls

(i) getpid() -

- getpid stands for Get the Process ID.
- The getpid() function shall return the process ID of the calling process.
- The getpid() function shall always be successful and no return value is reserved to indicate an error.

Example:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("PID: %d\n", getpid());
    printf("UID: %d\n", getuid());
    return 0;
}
```

(ii) getuid() -

- getuid stands for Get the User ID.
- The getuid() function shall return the User ID of the calling process.
- The getuid() function shall always be successful and no return value is reserved to indicate an error.

Example:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("PID: %d\n", getpid());
    printf("UID: %d\n", getuid());
    return 0;
}
```

(iii) gethostname() -

- gethostname stands for Get the hostname of the system.
- The gethostname() function shall always be successful and no return value is reserved to indicate an error.

Example:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    char name[100];
    gethostname(name, sizeof(name));
    printf("Hostname: %s\n", name);
    return 0;
}
```

(iv) sysinfo() -

- Returns system-wide information.

Example:

```
#include <sys/sysinfo.h>
#include <stdio.h>

int main() {
    struct sysinfo info;
    sysinfo(&info);
    printf("Uptime: %ld seconds\n", info.uptime);
    return 0;
}
```